

Comparative Analysis of Deep Learning Models for Real-World ISP Network Traffic Forecasting

Josef Koumar¹, Student Member, IEEE, Timotej Smoleň, Kamil Jeřábek², and Tomáš Čejka¹

Abstract—Accurate network traffic forecasting is crucial for Internet service providers to optimize resources, improve user experience, and detect anomalies. Until recently, the lack of large-scale, real-world datasets limited the fair evaluation of forecasting methods. The newly released CESNET-TimeSeries24 dataset addresses this gap by providing multivariate traffic data from thousands of devices over 40 weeks at multiple aggregation granularities and hierarchy levels. In this study, we leverage the CESNET-TimeSeries24 dataset to conduct a systematic evaluation of state-of-the-art deep learning models and provide practical insights. Moreover, our analysis reveals trade-offs between prediction accuracy and computational efficiency across different levels of granularity. Beyond model comparison, we establish a transparent and reproducible benchmarking framework, releasing source code and experiments to encourage standardized evaluation and accelerate progress in network traffic forecasting research.

Index Terms—Neural networks, deep learning, network traffic forecasting, network traffic prediction, network monitoring.

I. INTRODUCTION

TRAFFIC monitoring is a cornerstone of effective network management and cybersecurity, providing Internet Service Providers (ISPs) with critical insights to detect anomalies, mitigate congestion, and maintain network performance [1]. The surge in video streaming, cloud computing, and online gaming is driving rapid growth in Internet usage, contributing to increasingly complex and less predictable network traffic. Efficient network monitoring allows

ISPs to maintain service quality, mitigate security risks, and optimize bandwidth in real time [2].

However, real-time monitoring alone is insufficient for proactively managing network resources. To anticipate variations in demand and prevent service disruptions, ISPs increasingly adopt advanced forecasting techniques to predict traffic patterns, optimize resource allocation, scale network capacity, and sustain service quality under fluctuating loads [3]. The predictions can be integrated into Software-Defined Networking (SDN), Network Function Virtualization (NFV), and Service Level Agreement (SLA) frameworks [4], [5], [6], [7], allowing for programmable orchestration of network resources based on predicted traffic demands. For example, SDN controllers can leverage traffic forecasts to proactively adjust routing paths and traffic engineering policies, minimizing congestion and improving latency [4]. NFV environments benefit from forecasting by dynamically scaling Virtual Network Functions (VNFs) and optimally placing them across distributed cloud and edge data centers, thus enhancing service elasticity and reducing operational costs [5]. SLA monitoring benefits from predicting violations early and guiding automated resource allocation, which improves network performance [6], [7].

The rise of diverse, high-bandwidth services has significantly increased network traffic variability. Traditional models like ARIMA and exponential smoothing, which assume linearity, struggle with ISP data due to prevalent non-linear and high-frequency fluctuations, especially during peak traffic hours [8].

The adoption of deep learning models, particularly neural networks, has been driven by their ability to capture complex temporal dependencies across forecasting domains [9]. This capability significantly enhances network traffic forecasting and provides ISPs with deeper insights into usage trends and anomalies. However, Wu and Keogh [9] emphasize that evaluating recent advancements in forecasting methods remains difficult due to the lack of long-term real-world datasets. They show that relying on synthetic datasets can create the illusion of progress that does not generalize to real-world conditions. Similarly, Ferreira et al. [10] identify the absence of a reference dataset for network traffic forecasting as a major obstacle to performance evaluation. They note that most approaches are still tested only on artificially generated data or non-publicly available data.

The recent release of the CESNET-TimeSeries24 dataset [11] provides real-world ISP network traffic data from thousands of devices at multiple aggregation granularities,

Received 20 March 2025; revised 7 October 2025 and 13 November 2025; accepted 18 November 2025. Date of publication 25 November 2025; date of current version 29 December 2025. This work was supported by the Student Summer Research Program 2024 of FIT CTU in Prague. Furthermore, this research was partially funded by the Ministry of Interior of the Czech Republic in project “Flow-Based Encrypted Traffic Analysis” (VJ02010024), by the Ministry of Education, Youth and Sports of the Czech Republic in project “e-Infrastructure CZ” (LM2023054), and also by the Grant Agency of the CTU in Prague, grant No. SGS23/207/OHK3/3T/18 funded by the MEYS of the Czech Republic. Computational resources were provided by the e-INFRA CZ project (ID:90254), supported by the Ministry of Education, Youth and Sports of the Czech Republic. The associate editor coordinating the review of this article and approving it for publication was Y. Diao. (Corresponding author: Josef Koumar.)

Josef Koumar and Tomáš Čejka are with the Faculty of Information Technology, Czech Technical University in Prague, 160 00 Prague, Czech Republic, and also with the Administration and Security Tools Department, CESNET, a.i.e., 160 00 Prague, Czech Republic (e-mail: koumarj@fit.cvut.cz).

Timotej Smoleň is with the Faculty of Information Technology, Czech Technical University in Prague, 160 00 Prague, Czech Republic.

Kamil Jeřábek is with the Faculty of Information Technology, Brno University of Technology, 601 90 Brno, Czech Republic.

Digital Object Identifier 10.1109/TNSM.2025.3636557

hierarchical levels, and with various time series metrics. This dataset enables a level of comprehensive evaluation in network traffic forecasting that was previously impossible with limited or artificially generated datasets. Building on this, our study conducts an in-depth investigation of the performance and limitations of state-of-the-art deep learning models across diverse conditions and settings, while leveraging the unique features of CESNET-TimeSeries24. Using a common, reliable dataset allows for a direct comparison of the strengths and weaknesses of different forecasting models, offering critical insights for the development of forecasting frameworks and systems. Our investigation is guided by the following research questions:

RQ1: What is the performance of state-of-the-art deep learning models for real-world network traffic forecasting?

RQ2: How do the training and prediction windows impact the forecasting performance of state-of-the-art deep learning models?

RQ3: How does the granularity of aggregation (e.g., individual IP addresses, subnets, or institutions) impact the forecasting performance of state-of-the-art deep learning models?

RQ4: How do different monitored metrics (e.g., transmitted data, TCP / UDP ratio, average duration) affect the forecasting performance of state-of-the-art deep learning models?

The contribution of this study goes beyond comparing advanced forecasting models on a comprehensive dataset. It offers insights that are both model-specific and broadly relevant to the network forecasting domain, enabled for the first time by the availability of a large-scale real-world dataset. In addition, by combining rigorous methodological practices with open-access resources, we establish a fair, transparent, and reproducible benchmark for future research. Our goal is to encourage the adoption of standardized datasets, enabling consistent comparisons and generating practical insights. To further support reproducibility, we release the source code for all models and experiments on GitHub.¹

The rest of this paper is structured as follows. Section II describes the related literature. Section III describes the dataset used. Section IV describes selected state-of-the-art deep learning models for forecasting. Section V describes the methodology of the comparison analysis. Section VI describes the results. Section VII discusses the research questions and the lessons learned. Finally, Section VIII summarizes the paper.

II. RELATED WORK

Network traffic forecasting is a critical area of research due to its importance in network optimization, resource allocation, and anomaly detection [2], [3], [10]. Various approaches have been developed over the years, leveraging both statistical and machine learning methods [10].

Currently, statistically-based methods are commonly applied in network traffic forecasting with many published approaches. Such statistically-based methods include Auto-Regressive Integrated Moving Average (ARIMA), Seasonal ARIMA

(SARIMA), Exponential Smoothing (ETS), or Hidden Markov Models [12], [13], [14], [15]. However, there are several limitations of statistical-based forecasting [10], [16], [17]. Most of them rely on the assumption of linearity, which restricts their ability to model complex, non-linear relationships often present in real-world systems. Methods like ARIMA and regression models require stationarity in data, meaning trends and variance must remain constant over time, necessitating preprocessing that can complicate analysis. Moreover, these models are sensitive to parameter estimation, and their predictive accuracy can degrade in highly dynamic environments where relationships between variables fluctuate. Statistical approaches also struggle to handle multivariate interactions effectively, leading to oversimplified representations of intricate patterns. These limitations collectively underscore the challenges of applying statistical models in scenarios with complex, non-linear, or rapidly changing dynamics.

Therefore, deep learning has emerged in recent years as a transformative approach to network traffic forecasting. Primarily due to its ability to model non-linear relationships, capture complex dependencies within data, handle non-stationary time series effectively, and provide superior scalability [10], [18].

Deep learning became increasingly valuable for forecasting, particularly with the introduction of Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), which mitigate the exploding and vanishing gradient problems that affected earlier Recurrent Neural Networks (RNN) [19]. These deep learning models have been successfully applied in the network traffic forecasting domain [20], [21]. Nevertheless, the rapid expansion of deep learning forecasting variants in recent years underscores the need for a comprehensive evaluation.

The first major enhancement of LSTM and GRU models was their integration with Fully Convolutional Networks (FCNs) [22], [23]. Moreover, Temporal Convolutional Networks (TCNs) and their combination with an RNN variant were successfully applied in network traffic forecasting [24]. These hybrid models gained popularity due to their ability to outperform classical approaches while requiring minimal data preprocessing and easily scaling to large time series datasets [25].

Moreover, more sophisticated deep learning models originally developed for image classification have also been applied in the time series forecasting area. Notable examples include Residual Networks (ResNet) [26]. Building on these architectures, additional models have been developed, such as InceptionTime [27], which integrates residual blocks with Global Average Pooling layers.

Finally, one of the most recent forecasting approaches uses Transformers, which have become a prominent tool in time series forecasting due to their capability to model long-range dependencies, critical for network traffic prediction [28]. However, traditional Transformers face challenges such as inefficiency and a lack of interpretability in capturing temporal dynamics. To address these limitations, several advancements have been proposed. For instance, ETSformer [29] introduces exponential smoothing and frequency attention mechanisms to

¹<https://github.com/koumajos/isp-forecasting-benchmark>

replace conventional self-attention, enabling the model to capture both seasonal and trend components in a decomposable and interpretable manner. Similarly, models like Informer [30] and Autoformer [31] leverage sparsity and frequency-based approaches to enhance efficiency and adaptability in long-term sequence forecasting. Despite their success, recent studies question the necessity of complex Transformer-based architectures, with findings that simple linear models can outperform them in several scenarios [32]. Nevertheless, Transformers remain at the forefront of innovations in network traffic forecasting due to their ability to incorporate dynamic time-dependent features and extract meaningful patterns from large-scale, non-stationary data [33], [34].

The forecasting is usually done separately when multiple time series of multiple processes are present [35]. However, a recent study [36] also studied forecasting of groups of related time series, exploring how algorithms can leverage shared structure among series.

Many of the published works utilize different datasets, which makes approaches hardly comparable. For example, many datasets used in the network traffic forecasting domain were not created with the intention of forecasting evaluation, making them unsuitable for such purposes. An example of this is the WIDE project data in the MAWI archive [37], which contains only 15 minutes per day. Moreover, some related work are built on datasets that are not publicly available [38], [39], making it impossible to independently verify the results. Others rely on synthetic data [40], [41], which, while useful for controlled experiments, fail to provide insights into real-world network environments. Furthermore, even widely used datasets, such as KDD 99, NSL-KDD, Internet2 [42], GEANT [43], and Italia Telecom [44], may lack the comprehensive representation of modern, dynamic traffic patterns required for robust evaluation.

The recent release of a large, standardized, and publicly available real-world dataset, CESNET-TimeSeries24 [11], provides a valuable resource for comparing existing forecasting models. This comprehensive dataset enables a more consistent evaluation of deep learning approaches in realistic network scenarios. By leveraging this dataset, our study aims to perform a comparative analysis of state-of-the-art deep learning models, filling a crucial gap in the domain by establishing more reliable and meaningful performance assessment.

III. DATASET

To evaluate state-of-the-art deep learning models for real-world network traffic forecasting, we selected a dataset from an actual operational network. The CESNET-TimeSeries24 dataset [11] captures network traffic data from the ISP network CESNET3.²

The dataset is uniquely comprehensive, offering multivariate time series created through traffic aggregation at three distinct intervals: 10 minutes, 1 hour, and 1 day. Each interval-specific time series contains multiple metrics useful for network traffic forecasting and anomaly detection. Given that the original capture interval was 10 minutes, metrics that represent unique

TABLE I
AVAILABLE TIME SERIES METRICS FOR 1 HOUR AGGREGATION IN THE CESNET-TIMESERIES24 DATASET

Time series metric name	Description
n_flows	Number of IP flows
n_packets	Number of packets
n_bytes	Number of bytes
sum_n_dest_ip	Sum of number of unique destination IPs
avg_n_dest_ip	Avg. of number of unique destination IPs
std_n_dest_ip	Std of number of unique destination IPs
sum_n_dest_ports	Sum of number of unique destination ports
avg_n_dest_ports	Avg. of number of unique destination ports
std_n_dest_ports	Std of number of unique destination ports
sum_n_dest_asn	Sum of number of unique destination ASNs
avg_n_dest_asn	Avg. of number of unique destination ASNs
std_n_dest_asn	Std of number of unique destination ASNs
tcp_udp_ratio_packets	TCP/UDP packet ratio
tcp_udp_ratio_bytes	TCP/UDP bytes ratio
dir_ratio_packets	Directional ratio of packets
dir_ratio_bytes	Directional ratio of bytes
avg_duration	Average duration
avg_ttl	Average TTL

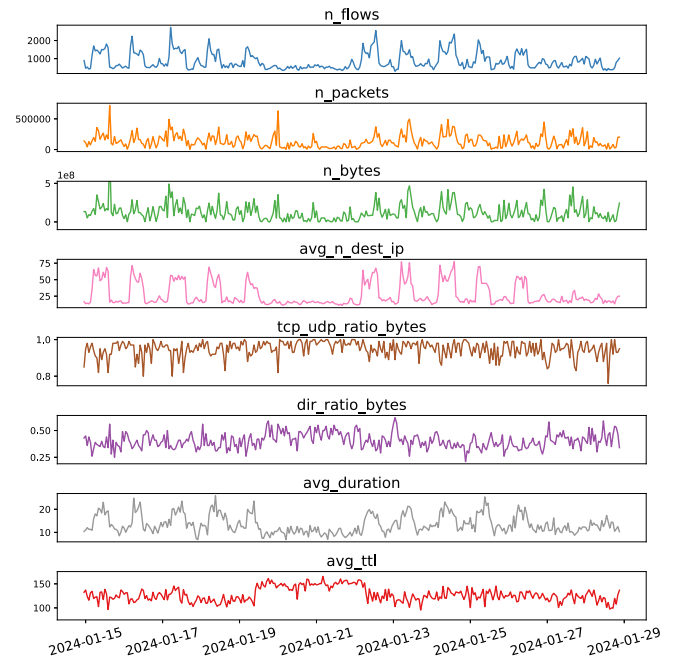


Fig. 1. Example of time series metrics aggregated by 1 hour from the CESNET-TimeSeries24 dataset.

values were adapted for the 1 hour and 1 day series incorporating sum, average and standard deviation statistics to enhance temporal granularity. We limit this study to 1 hour aggregation interval. The time series metrics for 1 hour aggregation interval are listed in Table I. The example multivariate time series aggregated by 1 hour is shown in Figure 1.

A standout feature of the CESNET-TimeSeries24 dataset is its extensive coverage and granularity. The dataset encompasses not only general network traffic but also highly detailed time series across 283 institutions, 548 institutional subnets, and over 270,000 individual IP addresses. This extensive range provides a robust basis for comparative analysis of deep learning models, enabling researchers to benchmark forecasting performance across multiple hierarchical levels within the network. This dataset enables a detailed examination of

²The Czech Educational and Science Network.

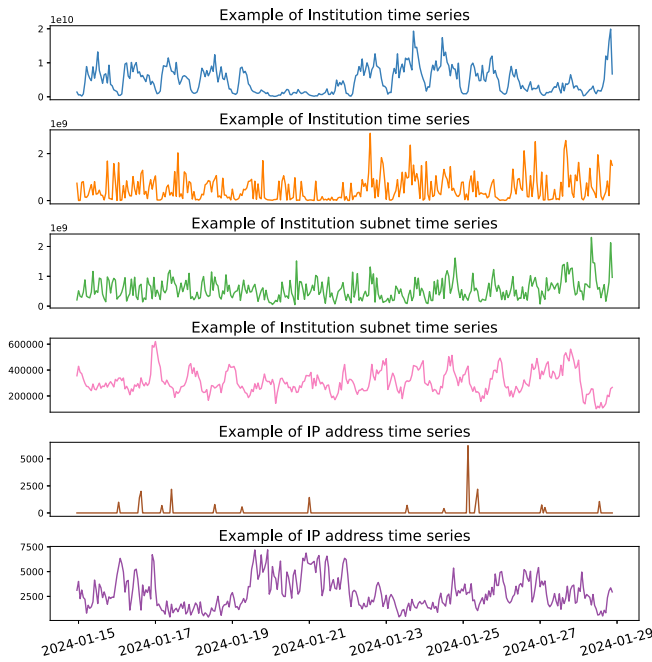


Fig. 2. Example of different hierarchical levels in time series metric *Number of bytes* aggregated by 1 hour from the CESNET-TimeSeries24 dataset.

the scalability, adaptability, and performance of models in forecasting diverse traffic patterns within a real-world ISP environment. Spanning a substantial 40-week period, from October 2023 to July 2024, the dataset captures both long-term trends and fine-grained fluctuations, making it suitable for comprehensive performance evaluation. The examples of different behaviors of time series of institutions, institution subnets, and IP addresses are shown in Figure 2.

IV. DEEP LEARNING MODELS FOR FORECASTING

This section provides a brief description of model types or particular state-of-the-art deep learning model architectures chosen to be compared in this study. The types were chosen to represent versatility in time series prediction handling tasks and to vary in computational demands.

Moreover, we provide information about their general advantages and shortcomings that can be found in related work. These models were selected due to their distinct architectural approaches, allowing for a comparative analysis of their strengths and limitations in different predictive contexts. Given the diverse requirements of real-time network operations, the models range from resource-intensive deep learning architectures to more streamlined alternatives, each offering unique benefits and trade-offs. Understanding these differences is essential to assessing the deployability of models in real-world network environments, where both computational efficiency and prediction accuracy are critical factors.

Mean model used as a baseline for evaluating deep learning models, computes the average value of data-points from the training window, and applies this mean as the prediction for each datapoint in the prediction window. While its simplicity is advantageous for benchmarking, it can capture only trends and fails to capture

temporal patterns and seasonality, making it unsuitable for datasets with significant time dependencies. Nevertheless, its role as a baseline is crucial, as it sets a minimal performance threshold that deep learning models should surpass, ensuring their predictive capacity is not inferior to a simplistic, non-adaptive approach.

SARIMA (Seasonal AutoRegressive Integrated Moving Average) is a statistical model for time series forecasting. SARIMA is interpretable and effective for data with strong linear trends and seasonal structures. However, due to reliance on linear assumptions, SARIMA cannot capture complex nonlinear relationships in data. SARIMA is used to compare deep learning models with a classical statistical approach.

LSTM (Long Short-Term Memory) networks are well-regarded for their capacity to capture long-term dependencies in sequence data, as they mitigate the vanishing gradient problem that hinders traditional RNNs. The architecture utilizes memory cells, which allow the model to retain information over extended time steps, making it highly effective for time-series prediction tasks. However, LSTMs are computationally intensive and require substantial memory resources, which can be a limitation in environments with constrained computational power. Additionally, they tend to have a large parameter count, which can lead to overfitting, especially when working with smaller datasets.

GRU (Gated Recurrent Unit) networks are a simplified alternative to LSTM networks, designed to efficiently capture dependencies in sequential data while addressing the vanishing gradient problem. GRUs streamline the architecture by combining the forget and input gates into a single update gate, which reduces the parameter count and computational demands. This makes GRUs faster to train and more memory-efficient, especially on smaller datasets or resource-constrained environments. However, their simplified structure may limit their ability to model complex long-term dependencies as effectively as LSTMs, which can be a drawback for tasks requiring extended temporal context.

LSTM-FCN model [22] combines the strengths of LSTM networks with Fully Convolutional Networks (FCNs), enabling the model to capture both temporal and spatial features in time-series data. This hybrid architecture improves feature extraction and helps generalize better to complex data patterns. Nevertheless, the increased complexity of combining LSTM with FCN layers results in greater computational demands and can make training slower, especially on larger datasets or limited hardware.

GRU-FCN model [23] integrates GRUs with Fully Convolutional Networks, blending the benefits of a simpler recurrent architecture with powerful feature extraction capabilities. GRUs offer a streamlined alternative to LSTMs, potentially reducing training time due to their more efficient structure. Nevertheless, GRU-FCN may not capture long-term dependencies with the same depth as LSTM models, which can

be a drawback when handling complex, long-sequence data.

ResNet models [26] are known for their residual connections, which help mitigate the vanishing gradient problem in deep neural networks, making them well-suited for extracting hierarchical features in time-series data. The residual connections enable training deeper architectures without the degradation issues typical in very deep networks. Nonetheless, ResNets can be resource-intensive and may require a significant amount of data to reach their full potential, as they are prone to overfitting on smaller datasets.

InceptionTime model [27] is a deep convolutional model inspired by the Inception architecture, specifically adapted for time-series classification. Its multi-scale feature extraction capabilities allow it to capture both local and global patterns in time-series data, contributing to its high performance across various benchmark tasks. However, the model's depth and complexity make it computationally expensive and may require extensive tuning to achieve optimal results, especially in cases with less structured or lower-dimensional data.

RCLSTM (Random Connectivity LSTM) model [45] is an optimized variant of LSTM designed to reduce computational complexity while maintaining comparable predictive performance. RCLSTM introduces stochastic connectivity to conventional LSTM neurons, creating a sparse neural structure that significantly decreases the number of trainable parameters. This modification improves computational efficiency, making the model particularly suitable for latency-stringent and power-constrained environments. Despite the reduction in connections, RCLSTM preserves long-range dependencies effectively, making it highly applicable in tasks such as traffic forecasting and user mobility prediction.

ETSformer model [29] is a deep learning model that integrates principles of Exponential Smoothing (ETS) into the Transformer architecture. By embedding trend and seasonal decomposition into the Transformer framework, ETSformer enhances interpretability while retaining the strengths of self-attention. This design allows it to capture both short-term fluctuations and long-term patterns more effectively. However, ETSformer's architecture is more complex than traditional methods, and training can be computationally demanding compared to other approaches.

The models and architectures discussed in this section are commonly used in the time series prediction domain, each providing promising robustness and modelling potential for this task, while varying in computational complexity. The variability in the chosen models should demonstrate the potential for their application in real-world network operations, where one has to balance between accuracy and computational efficiency.

V. METHODOLOGY

Our methodology follows the recommended usage of the CESNET-TimeSeries24 dataset [11]. We use multiple dataset parts provided in the dataset—institutions, institution subnets,

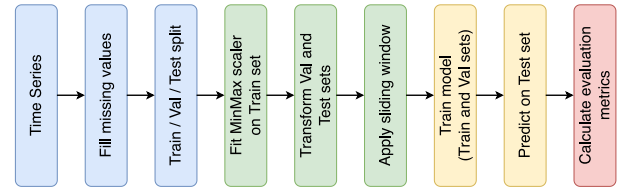


Fig. 3. Holistic view of methodology applied to each time series metric.

and an IP address sample. We trained models on time series aggregated on 1 hour time intervals, and each time series metric was modelled separately. Thus, we provide a comparison of only univariate approaches. We evaluate models on all time series metrics available in the dataset.

Pre-processing involves several steps for each time series, as shown in Figure 3, including data splitting, standardizing the data, and addressing missing values.

A. Missing Values Handling

Real-world network traffic is usually not constantly generated, especially in the case of monitoring a single device. Thus, it is inevitable that the collected time series contains missing values [46], which was identified as one of the challenges of the dataset [11].

In this comparative study, we fill missing values with zeros for most metrics. Since missing values in the dataset represent an interval where no data was transmitted. Thus, the zero value represents true information for metrics such as the number of IP flows, packets, and bytes. The zero values will force the model to learn how to predict when the device is not communicating. However, modeling time series that naturally contain zero values is a non-trivial challenge [47], [48]. Therefore, one of the objectives of this study is to verify how DL models handle time series with filled missing values. Because in the dataset, there are extreme cases where zero values represent a majority of values. The direction and TCP/UDP ratios are filled with 0.5 values, as they represent a true balance where no data is transmitted.

B. Time Series Splitting, Normalization, and Sliding Window

Each time series is split into train, validation, and test sets for the correct model training and evaluation. The duration of the training set represents the first 35% of the time series (14 weeks). The validation set holds the next 5% (2 weeks), and the remainder (24 weeks) is used as the test set.

At first, we fill in missing time series values with zeros. Then, we train the MinMax scaler on the train set and transform the train, validation, and test parts of the data using the fitted scaler.

We used the sliding-window approach, which is a widely adopted method for splitting data into multiple training samples from a single continuous time series stream. In our methodology, each neural network has an input of size equal to the number of consecutive time series values in a training window (also often referred to as a look-back window). The training window has a fixed length from which the model learns temporal dependencies. Based on this input, the model generates a specified number of consecutive future values. The

TABLE II
USED TRAINING AND PREDICTION WINDOWS

Training window size	Prediction window size
24	1
168	1
168	24
744	1
744	168

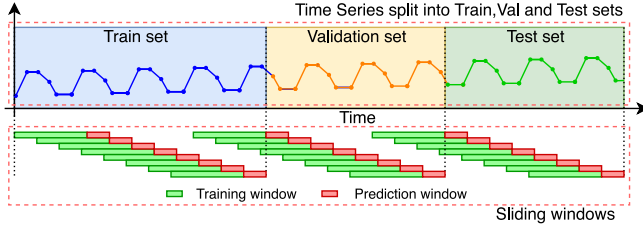


Fig. 4. Illustration of time series partitioning into training, validation, and test sets, along with the sliding window approach for generating overlapping training and prediction windows.

number of predicted future values depends on a predefined prediction window size (also often referred to as a forecast horizon).

The specific combinations of the training window size and the prediction window size evaluated in this study are summarized in Table II. We have chosen training and prediction window sizes commonly used in related work [10], [49]. In this comparative study, we systematically adjust the training and prediction window sizes to the day, week, and month. Thus, we explore how the performance of models varies depending on the amount of historical data considered and the length of the forecast.

Once a training–prediction pair is formed, the window is shifted forward by a prediction window. This process is repeated across the entire dataset, creating overlapping sequences that capture both short-term and long-term variations in the data. As a result, the model is trained on diverse input-output examples rather than relying on a single continuous segment. Thereby, it improves the ability of models to generalize.

In this study, the sliding window approach is applied consistently to the training, validation, and test sets of the time series, as illustrated in Figure 4.

C. Models Optimization

In this work, we are finding the optimal model architecture, which can perform best in general when considering all time series. Because varying hyperparameters specifically tuned for each separate time series is often inapplicable in real-world scenarios [50].

During the training of DL models, the Adam optimizer was employed. We choose the Adam due to its adaptive learning rate mechanism and momentum-based updates, which generally improve convergence speed and training stability compared to standard stochastic gradient descent [51].

We use MSELoss as the loss function to measure discrepancies between predicted and actual values. A hyperparameter

TABLE III
SUMMARY OF FINAL HYPERPARAMETERS

Model	RNN layers	Hidden size	Batch	Learning rate	Epochs
GRU	1	100	16	0.01	100
LSTM	1	100	16	0.01	100
GRU-FCN	4	20	16	0.01	20
LSTM-FCN	1	100	16	0.01	100
InceptionTime	-	-	128	0.001	20
ResNet	-	-	128	0.01	20
RCLSTM	1	300	32	0.01	100

search was conducted to explore various architectural and optimization configurations. This search includes the number of RNN layers, hidden layer size, batch size, learning rate, and number of epochs. The tested number of RNN layers and hidden size were chosen for each model based on related work that uses the model for network traffic prediction. Most of the models were trained with 1 to 5 RNN layers and hidden layer sizes from 20 to 200. We perform a search for optimal architecture on this scale of the number and sizes of layers. The final architecture and hyperparameter settings are presented in Table III.

These hyperparameters were selected based on the average results on a random sample of 50 time series from each dataset part. This setup of hyperparameter search allows finding a model setting that works generally well for network traffic prediction, regardless of the hierarchical level of aggregation or time series metric. Most of the models ended with one RNN layer, which may be surprising at first glance. However, it is in line with the findings of Zeng et al. [32]. In their work, they prove that a simple model can outperform even complicated models such as a Transformer-based model. For the LSTM-FCN and GRU-FCN models, we also tuned the sizes of CNN layers. Both model architectures contain three CNN layers with resulting best-performing sizes 128, 256, and 128 for LSTM-FCN, and 64-128-64 for GRU-FCN.

For the RCLSTM model, the architecture and hyperparameters were taken from the original publication, as their methodology also performs hourly forecasting. Likewise, InceptionTime, ResNet, and ETSformer were deployed with their standard, predefined architectures. By consolidating and refining the results of multiple experimental trials, the chosen hyperparameters perform best as evaluated across a range of time series in used datasets parts, as presented in the next section.

For the SARIMA model, we train parameters using the same methodology with all terms ranging from 1 to 5 and season ranging from one datapoint to one week of datapoints. The best performing model on the validation set of time series was $SARIMA(1,1,1)(1,1,1,1)$.

D. Evaluation Measures

The evaluation follows a common procedure where we forecast the next values based on previous ones using the trained models. To evaluate the precision of the forecasting, we use two common fully unsupervised measures, Root Mean Square Error (RMSE) and R^2 -score (also called the coefficient

of determination). In the following equations, y represents the actual normalized value, while $\hat{y}$ represents the predicted value. The RMSE is defined as:

$$\text{RMSE}(y, \hat{y}) = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} (y_i - \hat{y}_i)^2}. \quad (1)$$

The RMSE ranges from 0 to ∞ , with lower values indicating more accurate predictions. The measure highlights models that do not have large errors in prediction. However, its sensitivity to large errors can be insufficient, sometimes obscuring the distinction between consistent underperformance and occasional extreme errors.

The R^2 -score measure (also called the coefficient of determination) is a statistical measure that shows the proportion of the dependent variable's variance explained by the independent variable, calculated using the following equation:

$$R^2(y, \hat{y}) = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \mu_y)^2 + \epsilon}. \quad (2)$$

The R^2 -score ranges from $-\infty$ to 1, with higher values indicating more accurate predictions. The R^2 -score measure estimates goodness-of-fit, showing how well the model captures overall data trends.

Related work usually reports both RMSE and R^2 -score because of their distinguishing properties [52]. However, combining RMSE and R^2 -score measures offers a comprehensive evaluation of forecasting models by capturing different aspects of performance. Some studies combine various forecasting measures, for example, by harmonic mean [53]. RMSE emphasizes large individual errors and highlights variance, while the R^2 -score reflects how well the model explains the variance in the dependent variable.

To balance these contributions, we introduce a new combined measure that we call *Harmonic score*. This new measure combines the RMSE, giving insight into prediction errors, and the R^2 -score capturing trends, the two measures that we care about when forecasting network traffic. It uses the harmonic mean to penalize extreme values and ensure both metrics influence the final score similarly. The *Harmonic score* measure is defined as:

$$\text{Harmonic score} = 2 \frac{\text{RMSE} |R^2 - 1|}{\text{RMSE} + |R^2 - 1|}. \quad (3)$$

Since RMSE and R^2 -score operate on different scales, we rescale them by clipping RMSE at a maximum of 11 and R^2 -score at a minimum of -10 . These thresholds were chosen based on the distribution of model results. The distribution follows approximately a normal distribution. We choose $\mu + 2\sigma$ to remove outliers. Therefore, this captures the central 97.8% of the data to limit the influence of outliers while preserving typical values. This adjustment allows for a meaningful combination using the harmonic mean without distorting the overall evaluation.

This equation adjusts the R^2 -score by transforming it to align with RMSE, ensuring both measures are compatible. The harmonic mean emphasizes instances where both measures indicate precision, offering a clear, balanced evaluation.

VI. EXPERIMENTAL FORECASTING RESULTS

We evaluate the selected DL models across each time series metric and hierarchy level, leading to a large volume of results. However, due to space limitations, not all results can be presented in this paper. Therefore, we focus on presenting the results of the metric *Number of bytes* (transmitted data), as it is commonly used in the network traffic forecasting domain [10]. The results for the remaining features are presented briefly, focusing on key insights. All results and analyses are available in the supplementary materials on the GitHub repository.³

We trained 10 models (Mean, SARIMA, and 8 DL models) across five combinations of training and prediction windows, for 18 time series metrics within 1,831 time series (283 time series for institutions, 548 time series for institutional subnets, and 1000 time series for IP addresses), resulting in 1,647,900 final models. In this section, we present the results for the *Number of bytes* time series metrics (transmitted data).

A. Detailed Models Performance on Metric Number of Bytes

Tables IV and V present aggregated results for RMSE and R^2 -score respectively. Each row represents the mean of one experimental setting with a specific training and prediction window combination. The result is aggregated over all time series within the dataset time series type (e.g., Institutions, Institutions subnets, and single IPs time series). The best-performing model is highlighted with a bold font and green-shaded cell, while the worst-performing model is marked with a red-shaded cell. Lighter shades indicate second best or worst performance, providing a relative sense of each model's accuracy. If the values are very close or exactly the same within a row, it has the same shade.

Different evaluation metrics capture distinct aspects of forecasting quality. The resulting RMSE indicates that models such as GRU, LSTM, GRU-FCN, and LSTM-FCN perform similarly well with very close results in all scenarios. GRU and LSTM models slightly outperform their FCN-enhanced counterparts in R^2 -score better-matching the temporal trend of the data.

It can be seen that the results of the model performances depend on the window settings. For example, settings with a training window equal to or less than 168 values suggest that GRU-FCN and LSTM-FCN are quite effective in capturing overall trends in the data. Contrarily, the setting with a training window larger than 168 values suggests the opposite statement. The Mean and SARIMA do not perform well in the RMSE metric due to high prediction errors.

Mean model excels in terms of R^2 -score by accurately reflecting the general trend, although it lacks the capacity to model seasonal or cyclic variations. Among the deep learning models, RCLSTM demonstrates superior performance for one-step-ahead predictions of Institutional time series on R^2 -score. However, for the rest of the cases, it remains below average compared to the other models despite its promising superior performance described in [45]. The discrepancy is probably caused by different tested time series

³<https://github.com/koumajos/isp-forecasting-benchmark>

TABLE IV

OVERALL MEAN RMSE SHOWS THAT BASELINE MEAN AND SARIMA YIELD THE LARGEST ERRORS (LOWER IS BETTER), WHILE GRU, LSTM, GRU-FCN, AND LSTM-FCN PERFORM BEST. OTHER, MORE COMPLEX MODELS SHOW HIGHER ERRORS, WITH INCEPTIONTIME EVEN UNDERPERFORMING THE BASELINE IN SEVERAL CASES

	Training w.	Prediction w.	Mean	SARIMA	GRU	LSTM	GRU-FCN	LSTM-FCN	InceptionTime	ResNet	RCLSTM	ETSformer
Inst.	24	1	0.146	0.181	0.104	0.105	0.102	0.102	0.112	0.106	0.140	0.130
	168	1	0.149	0.144	0.105	0.106	0.104	0.103	0.124	0.127	0.140	0.129
	168	24	0.150	0.211	0.123	0.124	0.115	0.117	0.483	0.131	0.163	0.158
	744	1	0.151	0.135	0.106	0.106	0.111	0.110	0.136	0.146	0.112	0.112
	744	168	0.152	0.294	0.165	0.165	0.165	0.164	0.519	0.173	0.193	0.178
Mean			0.150	0.193	0.121	0.121	0.119	0.119	0.275	0.1366	0.149	0.144
Inst. subnets	24	1	0.383	0.407	0.218	0.219	0.217	0.217	0.230	0.220	0.323	0.338
	168	1	0.392	0.368	0.219	0.220	0.219	0.219	0.238	0.235	0.277	0.336
	168	24	0.394	0.447	0.237	0.237	0.229	0.231	0.590	0.244	0.334	0.373
	744	1	0.400	0.355	0.220	0.220	0.228	0.228	0.253	0.255	0.225	0.336
	744	168	0.402	0.574	0.265	0.265	0.269	0.265	0.613	0.275	0.326	0.399
Mean			0.394	0.430	0.231	0.232	0.232	0.232	0.384	0.245	0.297	0.356
IP addr.	24	1	1.010	1.127	0.149	0.150	0.150	0.151	0.165	0.152	0.221	0.958
	168	1	1.011	1.083	0.150	0.151	0.152	0.152	0.168	0.158	0.226	0.952
	168	24	1.011	1.197	0.154	0.154	0.154	0.158	0.573	0.159	0.247	0.976
	744	1	1.013	1.052	0.150	0.151	0.153	0.154	0.183	0.163	0.166	0.952
	744	168	1.014	1.090	0.179	0.179	0.214	0.189	0.593	0.182	0.234	0.980
Mean			1.010	1.109	0.156	0.157	0.165	0.161	0.336	0.163	0.219	0.964
Overall mean			0.518	0.578	0.169	0.170	0.172	0.171	0.332	0.182	0.222	0.488

TABLE V

OVERALL MEAN R^2 -score SHOWS BASELINE MEAN PERFORMS BEST, FOLLOWED BY GRU, LSTM, AND THEIR FCN VARIANTS, WHILE INCEPTIONTIME PERFORMS WORST

	Training window	Prediction window	Mean	SARIMA	GRU	LSTM	GRU-FCN	LSTM-FCN	InceptionTime	ResNet	RCLSTM	ETSformer
Inst.	24	1	0.093	-0.882	0.081	0.091	0.146	0.186	-0.773	0.058	0.161	-0.087
	168	1	0.040	0.023	0.030	0.101	0.167	0.144	-1.085	-0.630	0.159	0.062
	168	24	0.028	-1.022	-0.447	-0.451	-0.181	-0.285	-9.221	-0.733	-0.406	-0.602
	744	1	0.009	0.207	0.058	0.066	-0.046	-0.086	-1.586	-0.918	0.231	0.086
	744	168	-0.008	-2.668	-0.529	-0.493	-1.082	-0.541	-9.178	-0.737	-1.800	-1.280
Mean			0.032	-0.868	-0.161	-0.137	-0.199	-0.116	-4.369	-0.592	-0.331	-0.364
Inst. subnets	24	1	0.115	-1.130	-0.099	-0.046	0.063	0.079	-1.288	-0.150	0.062	-0.396
	168	1	0.062	-0.152	-0.098	-0.070	0.053	-0.013	-1.572	-0.678	0.073	-0.141
	168	24	0.047	-1.275	-0.548	-0.535	-0.268	-0.548	-8.903	-0.775	-0.657	-0.881
	744	1	0.021	0.126	-0.104	-0.087	-0.190	-0.236	-2.192	-1.054	0.104	-0.057
	744	168	-0.001	-3.453	-0.606	-0.551	-1.674	-0.938	-8.846	-0.915	-2.256	-1.530
Mean			0.049	-1.177	-0.291	-0.258	-0.423	-0.331	-4.560	-0.714	-0.535	-0.601
IP addr.	24	1	-0.001	-3.645	-0.464	-0.410	-0.124	-0.439	-2.698	-0.808	-0.089	-1.765
	168	1	0.013	-1.644	-0.409	-0.400	-0.177	-0.664	-2.818	-1.032	-0.118	-1.199
	168	24	0.009	-2.056	-0.308	-0.348	-0.612	-1.277	-9.223	-0.916	-0.988	-1.982
	744	1	-0.078	-0.618	-0.380	-0.393	-0.254	-0.623	-3.568	-1.167	-0.285	-0.846
	744	168	-0.087	-0.884	-0.505	-0.337	-3.968	-1.524	-9.169	-0.524	-2.775	-2.185
Mean			-0.029	-1.769	-0.413	-0.378	-1.027	-0.905	-5.495	-0.889	-0.851	-1.596
Overall mean			0.004	-1.271	-0.288	-0.258	-0.550	-0.451	-4.808	-0.732	-0.572	-0.974

of the original work and the time series from real-world network traffic. Finally, no single model consistently outperforms the others when predicting over longer horizons. The ETSformer, the most complex model based on transformers, achieves comparable performance with the leading models.

The SARIMA model is frequently used in the network traffic domain [10], [13], [54]. In our experiments, however, it shows weak performance, with high RMSE and low R^2 -scores. This outcome is largely due to our setup, which seeks a single best model configuration applicable across all time series in the dataset. SARIMA performs well only

TABLE VI
RESULTING MEAN *Harmonic Score* SHOWS THAT GRU, LSTM, AND THEIR FCN VARIANTS PERFORM BEST, WHILE MORE COMPLEX MODELS PERFORM WORSE

	Training window	Prediction window	Mean	SARIMA	GRU	LSTM	GRU-FCN	LSTM-FCN	InceptionTime	ResNet	RCLSTM	ETSformer
Inst.	24	1	0.146	0.213	0.120	0.121	0.115	0.116	0.134	0.122	0.119	0.128
	168	1	0.152	0.151	0.122	0.122	0.117	0.118	0.157	0.162	0.118	0.122
	168	24	0.153	0.243	0.156	0.157	0.141	0.144	0.850	0.171	0.162	0.170
	744	1	0.155	0.125	0.125	0.124	0.132	0.133	0.182	0.201	0.120	0.122
	744	168	0.156	0.392	0.171	0.171	0.171	0.169	0.855	0.186	0.224	0.206
Mean			0.152	0.225	0.139	0.139	0.135	0.136	0.436	0.169	0.149	0.150
Inst. subnets	24	1	0.177	0.292	0.152	0.154	0.148	0.150	0.174	0.155	0.180	0.155
	168	1	0.187	0.212	0.154	0.155	0.153	0.154	0.191	0.183	0.160	0.148
	168	24	0.189	0.291	0.188	0.188	0.173	0.177	0.868	0.202	0.219	0.199
	744	1	0.197	0.170	0.156	0.156	0.170	0.17	0.220	0.223	0.152	0.147
	744	168	0.202	0.526	0.203	0.201	0.210	0.202	0.872	0.222	0.270	0.244
Mean			0.190	0.298	0.171	0.171	0.171	0.170	0.465	0.197	0.196	0.179
IP addr.	24	1	0.300	0.852	0.129	0.131	0.131	0.132	0.159	0.134	0.164	0.151
	168	1	0.297	0.662	0.130	0.131	0.133	0.135	0.165	0.145	0.162	0.140
	168	24	0.298	0.702	0.138	0.138	0.137	0.146	0.938	0.148	0.175	0.189
	744	1	0.301	0.482	0.130	0.131	0.136	0.137	0.196	0.155	0.138	0.139
	744	168	0.303	0.424	0.151	0.150	0.221	0.170	0.943	0.156	0.193	0.200
Mean			0.300	0.624	0.136	0.136	0.152	0.144	0.480	0.148	0.166	0.164
Overall mean			0.214	0.382	0.148	0.149	0.152	0.150	0.460	0.171	0.170	0.163

when its hyperparameters are tuned for each individual series. However, this requirement is often impractical in real-world deployments [50]. Its performance also deteriorates with increasing proportions of zero values caused by device inactivity (see [11, Fig. 6]). In contrast, SARIMA performs reliably on time series without zero values, where it can predict the next data point based on longer historical patterns. Nevertheless, we utilize SARIMA only as a baseline for comparison in this work.

Table VI summarizes the forecasting performance using the Harmonic score, a metric that combines multiple evaluation aspects into a single measure. The results show that the GRU, LSTM, and LSTM-FCN architectures produce very similar results in several configurations. Despite the GRU-FCN model demonstrating superior performance in most experimental settings with a training period of 168 or less, it shows a significant drop in the score in the setting involving the IP address dataset, with a training window of 744 and a prediction window of 168. This decrease prevented GRU-FCN from outperforming other DL models in the overall mean.

Overall, classical architectures such as GRU, LSTM, and their convolutionally enhanced variants achieve similarly accurate results in terms of RMSE, R^2 -score, and Harmonic score. On the macro average, GRU-FCN performs best for the Institution dataset, LSTM-FCN for the Institution subnets dataset, and GRU for the IP address dataset. Considering the overall macro average across all datasets, GRU produces the most precise forecasts, followed by LSTM.

The transformer-based model ETSformer achieves an overall macro average result close to leading models. Mainly, the results below average on the IP address dataset part, causing the difference between ETSformer and the leading models.

We also observe that InceptionTime is the weakest performer across all evaluation measures, followed by ResNet. InceptionTime, originally designed for time series classification, struggles with network traffic forecasting due to its reliance on multi-scale convolutions, which increase model complexity without yielding significant performance gains. Its limited ability to capture long-term dependencies further hampers its accuracy. Similarly, ResNet, despite the use of residual connections, appears less effective in modeling sequential patterns, potentially leading to suboptimal generalization at high computational costs. These findings highlight the need for architectures better suited to network time series forecasting, such as those incorporating recurrent or attention-based mechanisms.

B. Factors Influencing Model Predictability

A clear performance trend emerges across the different dataset parts, reflecting the models' ability to capture data temporal trends based on the R^2 -score. Forecasting performance is highest for the Institution dataset, declines for Institution subnets, and is lowest for the IP address sample. This degradation likely stems from the increasing proportion of missing values, which are minimal in the Institution dataset but significantly more prevalent in the IP address sample. The impact of missing data is further supported by the strong negative correlation (-0.69 for the IP address dataset) between the ratio of missing samples and the R^2 -score.

Analyzing the most and least predictable time series reveals that less predictable series often display sudden, abnormal spikes in values, while the most predictable series follow more regular patterns. This observation is supported by

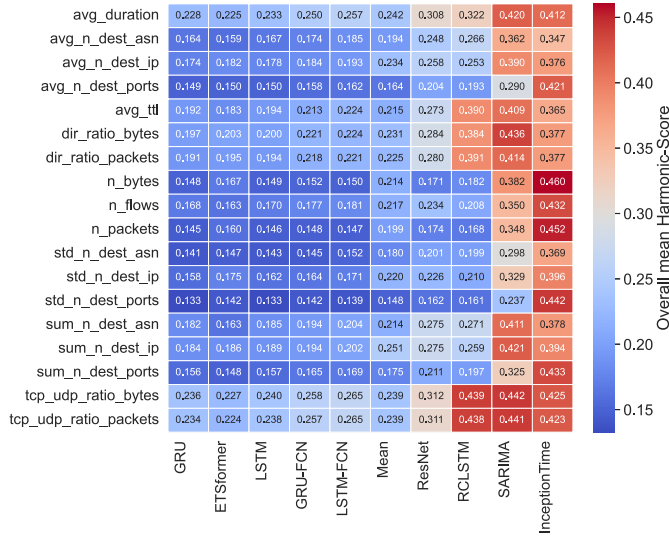


Fig. 5. Matrix providing results of overall mean *Harmonic score* for all time series metrics shows that the simpler models, including ETSformer, perform better than baseline Mean and SARIMA while leaving more complicated models behind.

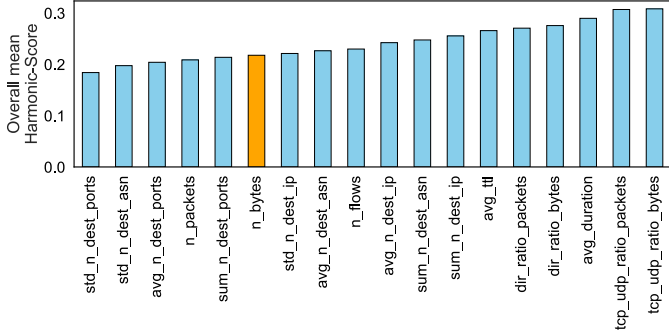


Fig. 6. Overall mean *Harmonic score* over all models for each time series metrics shows how easily predictable each time series is. The *Number of bytes* metric that is commonly utilized in network time series forecasting belongs to the less challenging ones.

autocorrelation and partial autocorrelation analyses, which show that highly predictable time series typically exhibit strong seasonal behavior.

In addition, the choice of training and prediction windows significantly affects model performance. Larger prediction windows generally degrade forecast accuracy, as models struggle to capture long-term dependencies with the same precision as short-term predictions.

C. Other Time Series Metrics

While the *Number of bytes* (i.e., transmitted data) is often used as a primary metric for network traffic forecasting, our results reveal that forecasting evaluation measures vary considerably across different metrics, as shown in Fig. 5 and Fig. 6. Some metrics follow similar predictability patterns to the *Number of bytes*, while others present additional challenges due to their inherent variability and dependence on specific network behaviors. Complete forecasting results for all metrics, along with code and supplementary analyses, are available on GitHub³.

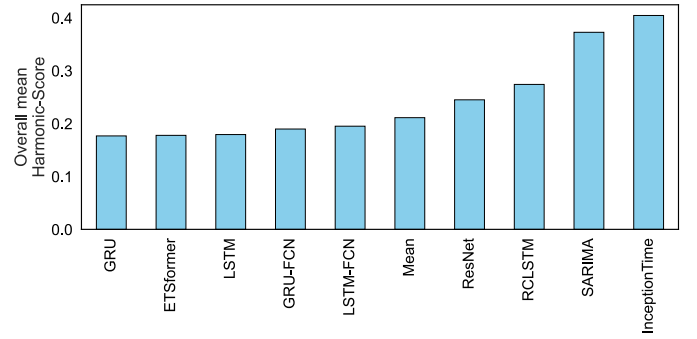


Fig. 7. Overall mean *Harmonic score* for each model shows how well the models perform among all evaluated metrics.

Metrics representing aggregate traffic volumes, such as the *Number of flows* and *Number of packets*, generally follow similar patterns to the *Number of bytes* and are effectively captured by DL models. These metrics often exhibit seasonal behavior, making them easier to predict, especially with recurrent architectures combined with convolutional layers. The top 5 DL models perform very similarly on these metrics.

In contrast, forecasting more dynamic and structurally complex metrics, such as *TCP/UDP ratios* and *Direction ratios*, proves significantly more challenging. These metrics are prone to sudden fluctuations due to external influences, such as changes in user behavior. Our results show that DL models struggle to capture these fluctuations, leading to a lower R^2 -score and increased forecasting error.

Overall mean result across all dataset parts and time series metrics is shown in Fig. 7. The results mostly follow the number of bytes. It can be seen that more structurally complex metrics are differentiators for the performance of models. Some models generally keep good performance regardless of the metric, while some are struggling to keep up. The ETSformer performs comparatively well and achieves the second-best position, even outcompeting other models in various metrics and measures.

We performed a statistical significance test to objectively validate the observed performance differences between models. We used a one-way ANOVA F-test, which compares the two best models (GRU and ETSformer models). The result of the ANOVA test is shown in Table VII. Statistically significant differences ($p < 0.05$) are consistently observed across most settings, confirming the robustness of our findings. However, for one experimental setting, there is no significant difference between GRU and ETSformer.

D. Deployability

Model deployability is a key factor; therefore, we measured the training and inference time of all DL models. The training was conducted on MetaCentrum⁴, which manages the National Grid Infrastructure (NGI) as part of the European Grid Infrastructure (EGI). To ensure consistency, all experiments were run on an identical pool of machines with fixed configurations. Each run used a dedicated virtual environment with 4

⁴<https://metavo.metacentrum.cz/en/index.html>

TABLE VII
RESULTS OF A ONE-WAY ANOVA F-TEST BETWEEN GRU AND ETSFORMER MODELS WITH $\alpha = 5\%$. THE P-VALUE IS ROUNDED TO 6 DECIMAL PLACES

	Training window	Prediction window	p value	Significant ($p < \alpha$)?
Inst.	24	1	0.828959	No
	168	1	0.000011	Yes
	168	24	< 0.000001	Yes
	744	1	< 0.000001	Yes
	744	168	0.000088	Yes
Overall			0.000003	Yes
Inst. subnets	24	1	< 0.000001	Yes
	168	1	< 0.000001	Yes
	168	24	< 0.000001	Yes
	744	1	< 0.000001	Yes
	744	168	0.000001	Yes
Overall			< 0.000001	Yes
IP addr.	24	1	< 0.000001	Yes
	168	1	0.006204	Yes
	168	24	< 0.000001	Yes
	744	1	0.000357	Yes
	744	168	< 0.000001	Yes
Overall			0.000001	Yes

TABLE VIII
RELATIVE TRAINING AND PREDICTION TIMES IN SECONDS
(NORMALIZED PER 100 DATAPOINTS)

	Training			Prediction		
	Inst.	Sub.	IP	Inst.	Sub.	IP
GRU	4.24	4.50	4.2	0.11	0.12	0.14
LSTM	5.32	4.27	4.50	0.10	0.11	0.10
GRU-FCN	1.19	0.95	1.04	0.06	0.05	0.05
LSTM-FCN	3.18	3.37	3.12	0.12	0.13	0.12
InceptionTime	2.93	3.01	2.95	0.07	0.07	0.08
ResNet	6.36	5.95	5.49	0.31	0.31	0.32
RCLSTM	11.37	8.44	16.28	0.30	0.28	0.28
ETSformer	181.14	176.22	160.26	3.35	3.28	3.28

exclusive CPUs, minimizing interference from other processes and ensuring comparable results.

Table VIII compares the relative training and prediction times of each model, normalized to 100 data points over all experiments. These metrics highlight the computational feasibility of each model for real-world deployment. Models with long training times may be impractical for frequent retraining, while high prediction latency can hinder responsiveness, particularly when forecasting a single next value. The results show that GRU-FCN consistently achieves the shortest training and prediction times across all dataset segments and experiments, making it a strong candidate for scenarios with limited time or computational resources. Moreover, the training and prediction times of ETSformer exceeded all other models by more than ten times. Therefore, ETSformer deployability is questionable, even with overall second-best results.

VII. DISCUSSION

In this section, we discuss key findings related to the four research questions (RQs) investigated in this study. We examine factors affecting forecasting precision, compare

the performance of state-of-the-art deep learning models on a public real-world network dataset, and highlight the practical implications of our results for network traffic forecasting.

RQ1: *What is the performance of state-of-the-art deep learning models for real-world network traffic forecasting?*

Our evaluation shows that state-of-the-art deep learning models can achieve competitive forecasting accuracy for real-world network traffic. However, no single model consistently outperforms others across all experimental settings, including dataset partitions, training windows, and prediction windows. The results suggest that while deep learning models can provide accurate forecasts, their effectiveness depends heavily on the specific experimental conditions.

GRU and LSTM architectures achieve the best overall results, while their hybrid variants with fully convolutional networks (GRU-FCN and LSTM-FCN) perform slightly better on the Institution and Institution subnets timeseries. Notably, GRU-FCN offers a balanced trade-off between accuracy and computational efficiency. In contrast, models like InceptionTime and ResNet underperform in most settings, possibly because their hierarchical feature extraction mechanisms are less suited for capturing the temporal dependencies and variability in network traffic data.

RQ2: *How do the training and prediction windows impact the forecasting performance of state-of-the-art deep learning models?*

The selection of training and prediction windows significantly impacts model accuracy, balancing the trade-off between capturing long-term dependencies and managing computational overhead. Short training windows often fail to capture temporal patterns beyond immediate fluctuations, leading to suboptimal performance. Conversely, excessively long training windows introduce noise and increase computational costs without substantial accuracy gains. However, our results show that the highest precision was achieved with small training windows, while increasing the window size led to a decline in performance.

For prediction windows, we observe a consistent trend: forecasting single next value yields significantly higher accuracy than longer prediction horizons. The degradation in accuracy over extended prediction windows is primarily due to error accumulation, a phenomenon observed across all evaluated DL models. Notably, LSTM and GRU exhibit the most stable performance across different prediction windows, highlighting their robustness in handling extended forecasts.

Our findings suggest that much of the network traffic time series lacks strong seasonality or contains large outliers, which models with short training and prediction windows handle more effectively. As a result, larger windows perform worse in this study. However, longer training and prediction windows could be advantageous in real-world deployments, as they require fewer inferences and are less reactive to abnormal spikes, making them potentially useful for anomaly detection.

RQ3: *How does the granularity of aggregation (e.g., individual IP addresses, subnets, or institutions) impact the*

forecasting performance of state-of-the-art deep learning models?

The forecasting precision of DL models is strongly influenced by the level of aggregation. Our results show that the ability to capture trend patterns is highest at the institution level, decreases at the subnet level, and is lowest at the individual IP address level.

This trend is primarily due to increasing data variability and sparsity at finer granularities. Institution-level traffic patterns are generally more stable and exhibit clear periodic trends, enabling higher predictive accuracy. In contrast, individual IP addresses often contain irregular spikes and a higher proportion of missing values, making them significantly harder to forecast. This aligns with our correlation analysis, which shows a strong negative relationship (-0.69) between missing data and R^2 -score at the IP address level. Models trained on IP-level data must contend with high noise levels and unpredictability, leading to increased forecasting errors.

These findings suggest that different modeling strategies may be required for different hierarchical levels. Institution-level forecasting may be well-suited to simpler recurrent models, while IP-level forecasting may require more advanced architectures or preprocessing techniques to handle sparsity and irregularities effectively.

RQ4: *How do different monitored metrics (e.g., transmitted data, TCP / UDP ratio, average duration) affect the forecasting performance of state-of-the-art deep learning models?*

Forecasting accuracy varies significantly across different network traffic metrics. Metrics with clear seasonal patterns, such as the number of flows, packets, and bytes, are well predicted by DL models. In contrast, highly variable metrics, such as TCP/UDP ratios, pose challenges due to their irregular fluctuations and external dependencies, resulting in lower forecasting precision. Aggregation level also plays a crucial role—metrics at the institution level are more predictable than those at the IP level due to reduced noise. Similarly, longer aggregation windows improve performance for volume-based metrics but may obscure short-term variations in ratio-based metrics.

Overall, DL models perform well on structured, high-volume metrics but struggle with fine-grained or bursty characteristics. Future research should explore adaptive strategies to improve forecasting for complex metrics, particularly assessing whether independently optimized DL models per metric outperform multivariate forecasting approaches.

VIII. CONCLUSION

In this study, we conducted a comprehensive evaluation of seven DL models for real-world network traffic forecasting. Our analysis revealed that LSTM and GRU surpass more complex architectures. Moreover, the GRU-FCN model achieved the best balance between predictive accuracy and computational efficiency. Beyond establishing a reproducible benchmark for network traffic forecasting using the real-world CESNET-TimeSeries24 dataset, our work lays the foundation for future work. Based on our findings, we mark several open challenges:

- 1) *Multi-time-series models:* Maintaining a separate model for each time series is both computationally expensive and operationally impractical. Future work should focus on developing a universal model capable of handling different instances within the same time-series aggregation without the need for retraining.
- 2) *Univariate vs. multivariate models:* Training separate models for each time series metric allows fine-tuning for specific behaviors but lacks scalability, requiring extensive computational resources and maintenance. Future work should explore the trade-offs between univariate and multivariate models for network traffic forecasting.
- 3) *Handling Heterogeneous Data:* Effectively managing large-scale, heterogeneous time series with high missing value rates is crucial. While imputing gaps with zeros can degrade model performance, it remains valuable for detecting outages and other critical events.
- 4) *Improving Interpretability:* Deep learning models often function as closed boxes, offering little insight into their decision-making process. Enhancing interpretability is crucial for resource allocation and anomaly detection, as it improves trust, facilitates debugging, and enables more informed decision-making.

REFERENCES

- [1] A. D'Alconzo, I. Drago, A. Morichetta, M. Mellia, and P. Casa, "A survey on big data for network traffic monitoring and analysis," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 3, pp. 800–813, Sep. 2019.
- [2] E. S. Hassan, A. E. A. Abdelaal, A. S. Oshaba, A. El-Emary, M. I. Dessouky, and F. E. A. El-Samie, "Optimizing bandwidth utilization and traffic control in ISP networks for enhanced smart agriculture," *PLoS ONE*, vol. 19, no. 3, 2024, Art. no. e0300650.
- [3] D. H. Oliveira, F. M. V. Filho, T. P. de Araújo, J. Celestino, and R. L. Gomes, "Adaptive model for network resources prediction in modern Internet service providers," in *Proc. IEEE ISCC*, 2020, pp. 1–6.
- [4] Y. Li and M. Chen, "Software-defined network function virtualization: A survey," *IEEE Access*, vol. 3, pp. 2542–2553, 2015.
- [5] T. Cucinotta et al., "Forecasting operation metrics for virtualized network functions," in *Proc. IEEE/ACM 21st Int. Symp. Cluster, Cloud Internet Comput. (CCGrid)*, 2021, pp. 596–605.
- [6] S. Shenoy, D. Gorinevsky, and N. Laptev, "Probabilistic modeling of computing demand for service level agreement," *IEEE Trans. Services Comput.*, vol. 12, no. 6, pp. 987–993, Nov./Dec. 2016.
- [7] B. Barua, F. A. Aurpa, M. Mubarrat, N. I. Protul, S. N. Howlader, and M. S. Kaiser, "AI for service-level agreement (SLA) optimization in microservices-based cloud infrastructures," in *Proc. 3rd Int. Conf. Sustain. Comput. Data Commun. (ICSCDS)*, 2025, pp. 485–492.
- [8] M. Ahmed, A. N. Mahmood, and J. Hu, "A survey of network anomaly detection techniques," *J. Netw. Comput. Appl.*, vol. 60, pp. 19–31, Jan. 2016.
- [9] R. Wu and E. J. Keogh, "Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 2421–2429, Mar. 2023.
- [10] G. O. Ferreira, C. Ravazzi, F. Dabbene, G. C. Calafiore, and M. Fiore, "Forecasting network traffic: A survey and tutorial with open-source comparative evaluation," *IEEE Access*, vol. 11, pp. 6018–6044, 2023.
- [11] J. Koumar, K. Hynek, T. Cejka, and P. Šiška, "CESNET-TimeSeries24: Time series dataset for network traffic anomaly detection and forecasting," *Sci. Data*, vol. 12, no. 1, p. 338, 2025.
- [12] H. Z. Moayed and M. A. Masnadi-Shirazi, "Arima model for network traffic prediction and anomaly detection," in *Proc. IEEE ITCC*, 2008, pp. 1–6.
- [13] V. C. Moussas, M. Daglis, and E. Kolega, "Network traffic modeling and prediction using multiplicative seasonal ARIMA models," in *Proc. IC-EPMSO*, 2005, pp. 6–9.
- [14] D. Tikunov and T. Nishimura, "Traffic prediction for mobile network using holt-winter's exponential smoothing," in *Proc. IEEE SoftCOM*, 2007, pp. 1–5.

- [15] Z. Chen, J. Wen, and Y. Geng, "Predicting future traffic using hidden Markov models," in *Proc. IEEE ICNP*, 2016, pp. 1–6.
- [16] M. Khashei and M. Bijari, "Which methodology is better for combining linear and nonlinear models for time series forecasting?" *J. Ind. Syst. Eng.*, vol. 4, no. 4, pp. 265–285, 2011.
- [17] M. P. Clements and D. F. Hendry, "On the limitations of comparing mean square forecast errors," *J. Forecast.*, vol. 12, no. 8, pp. 617–637, 1993.
- [18] M. Joshi and T. H. Hadi, "A review of network traffic analysis and prediction techniques," 2015, *arXiv:1507.05722*.
- [19] B. Lim and S. Zohren, "Time-series forecasting with deep learning: A survey," *Philosoph. Trans. Roy. Soc. A*, vol. 379, Apr. 2021, Art. no. 20200209.
- [20] A. Dalgkitis, M. Louta, and G. T. Karetos, "Traffic forecasting in cellular networks using the LSTM RNN," in *Proc. PCI*, 2018, pp. 28–33.
- [21] N. Ramakrishnan and T. Soni, "Network traffic prediction using recurrent neural networks," in *Proc. IEEE ICMLA*, 2018, pp. 187–193.
- [22] F. Karim, S. Majumdar, H. Darabi, and S. Chen, "LSTM fully convolutional networks for time series classification," *IEEE Access*, vol. 6, pp. 1662–1669, 2017.
- [23] N. Elsayed, A. S. Maida, and M. Bayoumi, "Deep gated recurrent and convolutional network hybrid model for univariate time series classification," 2018, *arXiv:1812.07683*.
- [24] J. Bi, X. Zhang, H. Yuan, J. Zhang, and M. Zhou, "A hybrid prediction method for network traffic with temporal convolutional network and LSTM," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 3, pp. 1869–1879, Jul. 2021.
- [25] F. Karim, S. Majumdar, and H. Darabi, "Insights into LSTM fully convolutional networks for time series classification," *IEEE Access*, vol. 7, pp. 67718–67725, 2019.
- [26] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.
- [27] H. Ismail Fawaz et al., "Inceptiontime: Finding alexnet for time series classification," *Data Min. Knowl. Discov.*, vol. 34, no. 6, pp. 1936–1962, 2020.
- [28] Q. Wen et al., "Transformers in time series: A survey," 2022, *arXiv:2202.07125*.
- [29] G. Woo, C. Liu, D. Sahoo, A. Kumar, and S. Hoi, "ETSformer: Exponential smoothing transformers for time-series forecasting," 2022, *arXiv:2202.01381*.
- [30] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 12, 2021, pp. 11106–11115.
- [31] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 22419–22430.
- [32] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in *Proc. AAAI*, 2023, pp. 1–8.
- [33] G. Kougiumtzidis, V. K. Poulkov, P. I. Lazaridis, and Z. D. Zaharis, "Mobile network traffic prediction using temporal fusion transformer," *IEEE Trans. Artif. Intell.*, vol. 6, no. 10, pp. 2685–2699, Oct. 2025.
- [34] Q. Kong et al., "Network traffic prediction: Apply the transformer to time series forecasting," *Math. Probl. Eng.*, vol. 2022, no. 1, 2022, Art. no. 8424398.
- [35] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*. Waterloo, ON, Canada: OTexts, 2018.
- [36] P. Montero-Manso and R. J. Hyndman, "Principles and algorithms for forecasting groups of time series: Locality and globality," *Int. J. Forecast.*, vol. 37, no. 4, pp. 1632–1653, 2021.
- [37] R. Fontugne, P. Borgnat, P. Abry, and K. Fukuda, "MAWILab: Combining diverse anomaly detectors for automated anomaly Labeling and performance benchmarking," in *Proc. ACM CoNEXT*, 2010, pp. 1–12.
- [38] Q. H. Do, T. T. H. Doan, T. V. A. Nguyen, N. T. Duong, and V. V. Linh, "Prediction of data traffic in telecom networks based on deep neural networks," *J. Comput. Sci.*, vol. 16, no. 9, pp. 1268–1277, 2020.
- [39] X. Zhou, Z. Zhao, R. Li, Y. Zhou, and H. Zhang, "The predictability of cellular networks traffic," in *Proc. IEEE ISCIT*, 2012, pp. 973–978.
- [40] M. Balanici and S. Pachnicke, "Multi-step forecasting of intense traffic streams using machine learning for optical circuit switching," in *Proc. ICTON*, 2019, pp. 1–4.
- [41] S. Li, J. Song, L. Xu, Y. Hu, W. Luo, and X. Zhou, "Network traffic prediction based on the feature of newly-generated network flows," in *Proc. IFIP Netw. Conf.*, 2022, pp. 1–8.
- [42] S. Uhlig, B. Quoitin, J. Lepropre, and S. Balon, "Providing public intradomain traffic matrices to the research community," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 36, no. 1, pp. 83–86, 2006.
- [43] A. Laghrissi and T. Taleb, "A survey on the placement of virtual resources and virtual network functions," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1409–1434, 2nd Quart., 2018.
- [44] G. Barlacchi et al., "A multi-source dataset of urban life in the city of Milan and the province of Trentino," *Sci. Data*, vol. 2, no. 1, pp. 1–15, 2015.
- [45] Y. Lei, W. Wang, T. Yan, N. Li, and A. Nandi, "Residual convolution LSTM network for machines remaining useful life prediction and uncertainty quantification," *J. Dyn., Monit. Diagn.*, vol. 1, pp. 2–8, Dec. 2021.
- [46] J. Koumar and T. Čejka, "Unevenly spaced time series from network traffic," in *Proc. TMA*, 2023, pp. 1–4.
- [47] G. O. Kaya, M. Sahin, and O. F. Demirel, "Intermittent demand forecasting: A guideline for method selection," *Sādāhāna*, vol. 45, no. 1, p. 51, 2020.
- [48] I. Svetunkov and A. Sroginis, "Why do zeroes happen? A model-based approach for demand classification," 2025, *arXiv:2504.05894*.
- [49] O. Aouedi, V. A. Le, K. Piamrat, and Y. Ji, "Deep learning on network traffic prediction: Recent advances, analysis, and future directions," *ACM Comput. Surv.*, vol. 57, no. 6, pp. 1–37, 2025.
- [50] J. Koumar, J. Pesek, K. Jerabek, and T. Čejka, "Towards building network outlier detection system for network traffic monitoring," in *Proc. IEEE Netw. Oper. Manag. Symp.*, 2025, pp. 1–6.
- [51] K. D. B. J. Adam and J. Ba, "A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [52] D. Chicco, M. J. Warrens, and G. Jurman, "The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation," *PeerJ Comput. Sci.*, vol. 7, p. e623, Jul. 2021.
- [53] M. W. Liemohn, A. D. Shane, A. R. Azari, A. K. Petersen, B. M. Swiger, and A. Mukhopadhyay, "RMSE is not enough: Guidelines to robust data-model comparisons for magnetospheric physics," *J. Atmos. Sol.-Terr. Phys.*, vol. 218, Jul. 2021, Art. no. 105624.
- [54] H. Yang, X. Li, W. Qiang, Y. Zhao, W. Zhang, and C. Tang, "A network traffic forecasting method based on SA optimized ARIMA-BP neural network," *Comput. Netw.*, vol. 193, Jul. 2021, Art. no. 108102.



Josef Koumar (Student Member, IEEE) is currently pursuing the Ph.D. degree with the Faculty of Information Technology, Czech Technical University in Prague. He is focusing on detecting threats and classifications of encrypted network traffic using time series analysis. He is also interested in anomaly detection. He is also working as a Researcher with CESNET a.l.e., the operator of the Czech national research and educational networks.



Timotej Smoleň received the master's degree from the Faculty of Information Technology, Czech Technical University in Prague. He is focusing on machine learning and big data analysis. He is part of the Laboratory of Network Traffic Monitoring, Faculty of Information Technology, Czech Technical University, Prague.



Kamil Jeřábek is currently pursuing the Ph.D. degree with the Faculty of Information Technology, Brno University of Technology. He is part of the Network and Distributed Systems Research Group. He has participated in several research projects, either national or international. His research interest is focused on topics related to computer networks, cybersecurity, big data, data analysis, and machine learning.



Tomáš Čejka is an Associate Professor with FIT CTU, Prague. He teaches network security course, and supervises a research group of bachelor/master/Ph.D. students. Meanwhile, he also works with CESNET a.l.e., the operator of the Czech national research and education network, as a Researcher and a Team Leader.